

SmartAdmin 《高质量代码思想 V3.0》

序

自2018年，发布《高质量代码思想V1.0》《Java规范V1.0》和《前端规范V1.0》，在这七年来，收到很多好评，截止目前已经有多家企业使用这套代码规范，同时也在结合大家的反馈、技术的发展、以及自身的不断成长中，一直保持不断迭代更新，同时也保证会一直更新！

高质量代码思想

- 我们推崇高质量的代码，身为开发，代码即利剑，键盘上一套行云流水，宛如侠客，事了拂衣去，深藏身与名。
- 我们推崇团队的高度配合默契、互相帮助，从不加班，而不是一看到别人的代码就头皮发麻，留其 [996.ICU](#) 加班。
- 我们始终坚持和坚信 生活是快乐的，工作也是快乐的，写代码则是更快乐的。
- 我们主动思考，保持谦逊，保持学习，热爱代码，更热爱生活。

前言

开源的代码规范基于阿里巴巴、华为的开发手册，添加了团队的风格和规范，补充了一些细节。感谢前人的经验和付出，让我们可以有机会站在巨人的肩膀上眺望星辰大海。

- 规范不是为了约束和禁锢大家的创造力，而是为了帮助大家能够在正确的道路上，尽可能的避免踩坑和跑偏。
- 规范可以让我们无论单枪匹马还是与众人同行的时候都能得心应手。
- 规范可以让我们在面对日益变态的需求和做代码接盘侠的时候，更优雅从容。
- 规则并不是完美的，通过约束和禁止在特定情况下的特性，可能会对代码实现造成影响。

我们制定规则的目的：为了大多数程序员小伙伴可以得到更多的好处，如果在团队实际运作中认为某个规则无法遵循或有更好的做法，希望大家可以共同改进该规范。

引自《阿里规约》的开头片段：现代软件架构的复杂性需要协同开发完成，如何高效地协同呢？无规矩不成方圆，无规范难以协同，比如，制订交通法规表面上是要限制行车权，实际上是保障公众的人身安全，试想如果没有限速，没有红绿灯，谁还敢上路行驶。对软件来说，适当的规范和标准绝不是消灭代码内容的创造性、优雅性，而是限制过度个性化，以一种普遍认可的统一方式一起做事，提升协作效率，降低沟通成本。代码的字里行间流淌的是软件系统的血液，质量的提升是尽可能少踩坑，杜绝踩重复的坑，切实提升系统稳定性，码出质量。

基础规范

1.1、什么是好的代码？

在之前的版本中，我们对好的代码引用了 Kent Beck 的简单设计四原则，大家反馈比较抽象。V3.0 决定用白话来简单讲讲好的代码原则：

- **满足业务需要：代码是来实现业务的，如果业务都实现不了，代码也就没什么价值了**
- **代码尽可能的清晰明了：就是让小白也能看懂你的代码**
- **代码尽可能的少：在保证清晰明了的前提下，能少一行少一行，能少一个类少一个类，能少一行注释少一行注释**
- **代码尽可能复用性和模块化：在保证清晰明了和尽可能少的前提下，能复用的代码尽量复用，能模块的尽量模块**

以上四个原则的重要程度依次降低，这是1024 创新实验室认为好代码的原则，即：简单的、好的、代码

1.2、英文单词命名规范

无论前端代码还是后端代码、异或其他代码，都是由一个个单词组成的，所以一个好的单词影响着代码的本身，所以定义如下：

1) 合理使用正确的英文单词

很多人认为自己英语不好就命名比较随意，但，一个有道词典或者百度翻译就能解决这件事情，所以单词的命名必须使用正确。曾经遇到过这种起名字的，比如有一个双11业务，要求第一天业务、第二天、第三天业务的区别，有些小伙伴这样起名字：

```
1    第一天： di1day
2    第二天： di2day
3    第三天： treeday    （三应该是 three，笑喷）
```

以上是真实发生的例子，故定义如下：

- 一定要用英文，且单词正确，不要用汉语拼音（特殊情况除外，比如某些税务系统的特殊科目命名）
- 英文单词一定要使用常用词
- 英文单词要符合业务

2) 合理区分名词和动词

在大部分编程语言中，项目、类Class、数据库、表名、url中的前半部分 等等 一个比较大的范围都是应该用名词，比如java中的 `class` 、 `interface` 、 `enum` 等都是名词，比如 `OrderService` 。

而具体的方法名应该是**动词**或**动名词**，比如方法：创建订单： `createOrder` ，查询订单 `queryOrder`

3) 各个端、数据库、等命名要统一

无论团队里的前端、后端、移动端、数据库、服务器、redis、docker等等一切对于 某个业务或者某个业务单元的命名必须高度保持一致。

```
1    比如之前遇到过这么一个功能，OA办公系统的`通知`功能，各个端定义为：
2    - 后端：`notice`，（NoticeController，表：t_notice）
3    - 前端用成了`news`，（news-list.vue， /news/news-list）
4    - 移动端用成了`message`，（MessageFragement）
5    - 最后再对接的时候，懵逼了，懵了；
```

1.3、注释规范

1) 注释和代码一样重要

注释是披荆斩棘历经磨难翻越需求这座大山时，留下的踪迹和收获的经验教训，这些宝贵的知识除了证明曾经存在过，也提醒着后来的人们殷鉴不远、继往开来。

注释除了说明作用、逻辑之外。还有一个很重要的原因：当业务逻辑过于复杂，代码过于庞大的时候，注释就变成了一道道美化环境、分离与整理逻辑思路的路标。这是很重要的一点，它

能有效得帮助免于陷入代码与业务逻辑的泥沼之中。

正例：

java

```
1  /**
2   * 开始抽奖方法
3   * 保存中奖信息、奖励用户积分等
4   * @param luckDrawDTO
5   * @return ResponseDTO 返回中奖信息
6   */
7  public ResponseDTO<String> startLuckDraw(LuckDrawDTO luckDrawDTO) {
8
9      // ----- 1、校验抽奖活动基本信息 -----
10     xxx伪代码一顿操作
11
12     // ----- 2、新增抽奖记录 -----
13     xxx伪代码一顿操作
14
15     // ----- 3、如果需要消耗积分，则扣除钢镚积分 -----
16     xxx伪代码一顿操作
17
18     // ----- 4、获取奖品信息，开始翻滚吧 -----
19     xxx伪代码一顿操作
20
21     return ResponseDTO.succ(luckDrawPrizeVO);
22 }
```

2) 注释和代码的一致性

注释并不是越多越好，当注释过多，维护代码的同时，还需要维护注释，不仅变成了一种负担，也与当初添加注释的初衷背道而驰。

首先：大家应该通过清晰的逻辑架构，好的变量命名来提高代码可读性；需要的时候，才辅以注释说明。注释是为了帮助阅读者快速读懂代码，所以要从读者的角度出发，按需注释。注释内容要简洁、明了、无二义性，信息全面且不冗余。

其次：无论是修改、复制代码时，都要仔细核对注释内容是否正确。只改代码，不改注释是一种不文明行为，破坏了代码与注释的一致性，会让阅读者迷惑、费解，甚至误解。

反例：

```
1 // 查询部门
2 EmployeeVO employee = employeeDao.listByDepartmentId(deptId);
```

3) 方法注释

方法要尽量通过方法名自解释，不要写无用、信息冗余的方法头，不要写空有格式的方法头注释。

方法头注释内容可选，但不限于：功能说明、返回值，用法、算法实现等等。尤其是对外的方法接口声明，其注释，应当将重要、有用的信息表达清楚。

正例：

java

```
1 /**
2  * 解析转换时间字符串为 LocalDate 时间类
3  * 调用前必须校验字符串格式 否则可能造成解析失败的错误异常
4  *
5  * @param dateStr 必须是 yyyy-MM-dd 格式的字符串
6  * @return LocalDate
7  */
8 public static LocalDate parseYMD(String dateStr){}
```

反例：

java

```
1 /**
2  * 校验对象
3  *
4  * @param t
5  * @return String
6  */
7 public static <T> String checkObj(T t);
```

反例中出现的问题：

- 方法注释没有说明具体的作用、使用事项。
- 参数、返回值，空有格式没内容。这是非常重要一点，任何人调用任何方法之前都需要知道方法对参数的要求，以及返回值是什么。

1.4、TODO FIX规范

TODO/TBD(to be determined) 注释一般用来描述已知待改进、待补充的修改点,并且加上作者名称。

FIXME 注释一般用来描述已知缺陷, 它们都应该有统一风格, 方便文本搜索统一处理。如:

```
1 // TODO <author-name>: 补充XX处理
2 // FIXME <author-name>: XX缺陷
```

java

举例:

```
1 // TODO <卓大> 为了数据安全, 此处应该对手机号进行加*处理
```

js

1.5、无用代码：删！

因为现在所有的项目都使用了代码管理工具, 比如 git、svn、ts等等, 所以对于无用的代码, 让我们尽情的删除掉吧!

重要的说三遍:

不要注释, 不要注释, 不要注释!

要删除代码, 要删除代码, 要删除代码!

1.6、代码Git提交规范

- 提交前应该冷静、仔细检查一下, 确保没有忘记加入版本控制或不应该提交的文件。
- 提交前应该先编译一次 (idea 里 ctrl+F9), 防止出现编译都报错的情况。
- 提交前先更新 pull 一次代码, 提交前发生冲突要比提交后发生冲突容易解决的多。
- 提交前检查代码是否格式化, 是否符合代码规范, 无用的包引入、变量是否清除等等。
- 提交时检查注释是否准确简洁的表达出了本次提交的内容。
- 提交代码时必须填写详细备注, 如完成功能, 注释为“新增 XX 功能”;
- 若此次提交代码对应禅道中的任务或者 bug, 格式如下:

```
1 task#[任务id] [任务标题] [具体事项]
2 bug#[bug id] [bug标题] [具体事项]
```

- 例子如下:

```
1 git commit -m 'task#1101 开发smartreload功能 完成线程池的编码'
2 git commit -m 'bug#1102 smartreload时间不正确 线程池的大小问题'
```

1.7、保持项目整洁

使用 git，必须添加 .gitignore 忽略配置文件。
不要提交与项目无关的内容文件：idea 配置、target 包等。

联系我们

[1024创新实验室-主任：卓大](#)，混迹于各个技术圈，研究过计算机，熟悉点 java，略懂点前端。
[1024创新实验室](#)，卓大的软件公司，致力于成为中原领先、国内一流的技术团队，以AI+数字化为驱动，用技术为产业互联网提供无限可能，业务如下：

- 供应链（网络货运、大宗贸易进销存ERP、物流TMS、B2B电商、仓储WMS、AI提效等）
- 教育（就业创业大数据平台、继续教育平台、在线教育系统、题库等）
- AI+软件（软件定制外包、数据大屏、国产化改造、人员外包、技术顾问、技术培训等）
- 欢迎各类合作哦~

加微信：卓大 拉你入群，一起学习	公众号：六边形工程师 分享：赚钱、代码、生活	“写好代码少加班，快乐工作，热爱生活，感谢支持。” 卓大的赞赏码	@六边形工程师 抖音号：1024lab 卓大，1024创新实验室主任；
		请“1024创新实验室” 烩面里加肉 咖啡配胡辣汤,提神又饱腹	抖音：六边形工程师 直播：赚钱、代码、中医

下一篇
[Vue3规范 V3.0](#)

